

Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «УрФУ имени первого Президента России Б.Н. Ельцина»
Институт радиоэлектроники и информационных технологий – РТФ

«Введение в многопоточность»

Лабораторное задание №6

Группа: РИМ-150950

Студент: Владимиров Н.А.

Преподаватель: Архипов Н. А.

Екатеринбург

2026

1. Цель работы

Получить представление о механизме многопоточности в языке программирования Java, научиться создавать и управлять потоками, использовать методы `sleep()`, `join()`, `wait()`, `notifyAll()`, а также применять многопоточность для обработки массивов.

2. Описание задачи

В разделе 1 методички приведены примеры решения задач:

1. запуск двух потоков, выводящих имя и текущее время в течение 10 секунд;
2. запуск потока, выводящего числа от 1 до 10 с задержкой 1 секунда;
3. два потока выводят чётные и нечётные числа от 1 до 10 с синхронизацией.

В разделе 2 необходимо реализовать задания для самостоятельной работы:

1. два потока выводят имя и время (аналог примера 1);
2. поток выводит числа от 1 до 10 (аналог примера 2);
3. два потока выводят чётные и нечётные числа (аналог примера 3);
4. создание 10 потоков, каждый выводит свой номер;
5. поиск максимального элемента в массиве с помощью многопоточности;
6. суммирование элементов массива с помощью многопоточности;
7. решение двух задач с сайта `timus`

3. Ход выполнения

Создан Java-проект lab_6. Примеры из раздела 1 размещены в пакете examples_lr6 (классы Example1–Example3). Задания 1–6 реализованы в пакетах lr6.task1–lr6.task6. Общие функции параллельной обработки массивов вынесены в класс ArrayParallelUtils. Запуск всех примеров и заданий выполняется через класс Main.

Репозиторий с исходным кодом: https://github.com/n0vision/Java_Labs_2026

3.1. Примеры решения задач (раздел 1)

Пример 1 (класс examples_lr6.Example1):

```
package examples_lr6;

import lr6.ThreadDemoConfig;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Example1 {

    private static final DateTimeFormatter TIME_FORMAT =
        DateTimeFormatter.ofPattern("HH:mm:ss");

    public static void run() throws InterruptedException {
        int durationSeconds = ThreadDemoConfig.getNameAndTimeDurationSeconds();
        long delayMs = ThreadDemoConfig.getNumberPrintDelayMs();

        Thread t1 = new Thread(createRunnable("t1", durationSeconds, delayMs),
            "t1");
        Thread t2 = new Thread(createRunnable("t2", durationSeconds, delayMs),
            "t2");

        t1.start();
        t2.start();
        t1.join();
        t2.join();
    }

    private static Runnable createRunnable(String name, int durationSeconds,
        long delayMs) {
        return () -> {
            for (int i = 0; i < durationSeconds; i++) {
                System.out.println(name + " " +
                    LocalDateTime.now().format(TIME_FORMAT));
                try {
                    Thread.sleep(delayMs);
                } catch (InterruptedException e) {

```

```

        Thread.currentThread().interrupt();
        return;
    }
}

};

}

public static void main(String[] args) throws InterruptedException {
    System.out.println("Пример 1. Два потока выводят имя и время");
    run();
}
}

```

Пример 2 (класс examples_lr6.Example2):

```

package examples_lr6;

import lr6.ThreadDemoConfig;

public class Example2 {

    public static void run() throws InterruptedException {
        long delayMs = ThreadDemoConfig.getNumberPrintDelayMs();

        Thread t = new Thread(() -> {
            for (int i = 1; i <= 10; i++) {
                System.out.println(i);
                try {
                    Thread.sleep(delayMs);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
        }, "t");

        t.start();
        t.join();
    }

    public static void main(String[] args) throws InterruptedException {
        System.out.println("Пример 2. Поток выводит числа от 1 до 10");
        run();
    }
}

```

Пример 3 (класс examples_lr6.Example3):

```

package examples_lr6;

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class Example3 {
    private static final ReentrantLock LOCK = new ReentrantLock();

```

```

private static final Condition CONDITION = LOCK.newCondition();
private static int number = 1;

public static void run() throws InterruptedException {
    number = 1;

    Thread evenThread = new Thread(Example3::runEven, "evenThread");
    Thread oddThread = new Thread(Example3::runOdd, "oddThread");

    evenThread.start();
    oddThread.start();
    evenThread.join();
    oddThread.join();
}

private static void runEven() {
    LOCK.lock();
    try {
        while (number <= 10) {
            if (number % 2 == 0) {
                System.out.println("Чётное: " + number);
                number++;
                CONDITION.signalAll();
            } else {
                CONDITION.await();
            }
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    } finally {
        LOCK.unlock();
    }
}

private static void runOdd() {
    LOCK.lock();
    try {
        while (number <= 10) {
            if (number % 2 != 0) {
                System.out.println("Нечётное: " + number);
                number++;
                CONDITION.signalAll();
            } else {
                CONDITION.await();
            }
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    } finally {
        LOCK.unlock();
    }
}

public static void main(String[] args) throws InterruptedException {
    System.out.println("Пример 3. Чётные и нечётные числа в двух потоках");
    run();
}
}

```

3.2. Задания для самостоятельной работы

Задание 1 (класс lr6.task1.Task1):

```
package lr6.task1;

import examples_lr6.Example1;

public class Task1 {
    public static void main(String[] args) throws InterruptedException {
        System.out.println("Задание 1. Два потока выводят имя и время");
        Example1.run();
    }
}
```

Задание 2 (класс lr6.task2.Task2):

```
package lr6.task2;

import examples_lr6.Example2;

public class Task2 {
    public static void main(String[] args) throws InterruptedException {
        System.out.println("Задание 2. Поток выводит числа от 1 до 10");
        Example2.run();
    }
}
```

Задание 3 (класс lr6.task3.Task3):

```
package lr6.task3;

import examples_lr6.Example3;

public class Task3 {
    public static void main(String[] args) throws InterruptedException {
        System.out.println("Задание 3. Чётные и нечётные числа в двух потоках");
        Example3.run();
    }
}
```

Задание 4 (класс lr6.task4.Task4):

```
package lr6.task4;

public class Task4 {

    public static void run() throws InterruptedException {
        Thread[] threads = new Thread[10];

        for (int i = 0; i < threads.length; i++) {
```

```

        int threadNumber = i + 1;
        threads[i] = new Thread(() -> System.out.println("Поток №" +
threadNumber), "thread-" + threadNumber);
        threads[i].start();
    }

    for (Thread thread : threads) {
        thread.join();
    }
}

public static void main(String[] args) throws InterruptedException {
    System.out.println("Задание 4. Десять потоков выводят свой номер");
    run();
}
}

```

Задание 5 (класс lr6.task5.Task5):

```

package lr6.task5;

import lr6.ArrayParallelUtils;

import java.util.Arrays;

public class Task5 {

    public static int findMax(int[] array) throws InterruptedException {
        return ArrayParallelUtils.parallelMax(array);
    }

    public static void main(String[] args) throws InterruptedException {
        int[] numbers = {12, -4, 87, 5, 33, 87, 0, -19, 44, 7};
        int max = findMax(numbers);

        System.out.println("Задание 5. Максимальный элемент массива");
        System.out.println("Массив: " + Arrays.toString(numbers));
        System.out.println("Количество потоков: " +
Runtime.getRuntime().availableProcessors());
        System.out.println("Максимум: " + max);
    }
}

```

Задание 6 (класс lr6.task6.Task6):

```

package lr6.task6;

import lr6.ArrayParallelUtils;

import java.util.Arrays;

public class Task6 {

    public static long sumArray(int[] array) throws InterruptedException {
        return ArrayParallelUtils.parallelSum(array);
    }
}

```

```

    }

    public static void main(String[] args) throws InterruptedException {
        int[] numbers = {12, -4, 87, 5, 33, 87, 0, -19, 44, 7};
        long sum = sumArray(numbers);

        System.out.println("Задание 6. Сумма элементов массива");
        System.out.println("Массив: " + Arrays.toString(numbers));
        System.out.println("Количество потоков: " +
Runtime.getRuntime().availableProcessors());
        System.out.println("Сумма: " + sum);
    }
}

```

3.3. Класс ArrayParallelUtils

```

package lr6;

import java.util.concurrent.CountDownLatch;

public final class ArrayParallelUtils {
    private ArrayParallelUtils() {}

    public static int parallelMax(int[] array) throws InterruptedException {
        if (array == null || array.length == 0) {
            throw new IllegalArgumentException("Массив не должен быть пустым");
        }

        int threadCount = Runtime.getRuntime().availableProcessors();
        int chunkSize = (array.length + threadCount - 1) / threadCount;
        int[] partialMax = new int[threadCount];
        CountDownLatch latch = new CountDownLatch(threadCount);

        for (int threadIndex = 0; threadIndex < threadCount; threadIndex++) {
            int start = threadIndex * chunkSize;
            int end = Math.min(start + chunkSize, array.length);
            int index = threadIndex;

            if (start >= array.length) {
                partialMax[index] = Integer.MIN_VALUE;
                latch.countDown();
                continue;
            }

            new Thread(() -> {
                int localMax = array[start];
                for (int i = start + 1; i < end; i++) {
                    localMax = Math.max(localMax, array[i]);
                }
                partialMax[index] = localMax;
                latch.countDown();
            }, "max-worker-" + threadIndex).start();
        }

        latch.await();
    }
}

```

```

        int result = partialMax[0];
        for (int i = 1; i < threadCount; i++) {
            result = Math.max(result, partialMax[i]);
        }
        return result;
    }

    public static long parallelSum(int[] array) throws InterruptedException {
        if (array == null) {
            throw new IllegalArgumentException("Массив не должен быть null");
        }
        if (array.length == 0) {
            return 0L;
        }

        int threadCount = Runtime.getRuntime().availableProcessors();
        int chunkSize = (array.length + threadCount - 1) / threadCount;
        long[] partialSum = new long[threadCount];
        CountDownLatch latch = new CountDownLatch(threadCount);

        for (int threadIndex = 0; threadIndex < threadCount; threadIndex++) {
            int start = threadIndex * chunkSize;
            int end = Math.min(start + chunkSize, array.length);
            int index = threadIndex;

            if (start >= array.length) {
                partialSum[index] = 0L;
                latch.countDown();
                continue;
            }

            new Thread(() -> {
                long localSum = 0L;
                for (int i = start; i < end; i++) {
                    localSum += array[i];
                }
                partialSum[index] = localSum;
                latch.countDown();
            }, "sum-worker-" + threadIndex).start();
        }

        latch.await();

        long result = 0L;
        for (long value : partialSum) {
            result += value;
        }
        return result;
    }
}

```

3.4. Класс Main

```

package lr6;

public class Main {
    public static void main(String[] args) throws InterruptedException {

```

```

        System.out.println("=== Лабораторная работа №6: Многопоточность ===\n");

        System.out.println("--- Примеры решения задач ---\n");
        examples_lr6.Example1.main(args);
        System.out.println();

        examples_lr6.Example2.main(args);
        System.out.println();

        examples_lr6.Example3.main(args);
        System.out.println();

        System.out.println("--- Задания для самостоятельной работы ---\n");
        lr6.task1.Task1.main(args);
        System.out.println();

        lr6.task2.Task2.main(args);
        System.out.println();

        lr6.task3.Task3.main(args);
        System.out.println();

        lr6.task4.Task4.main(args);
        System.out.println();

        lr6.task5.Task5.main(args);
        System.out.println();

        lr6.task6.Task6.main(args);
    }
}

```

3.5. Задачи Timus

Задача 1082: вычисление суммы чисел от 1 до 10.

```

package timus.task_1082;

public class Main {
    public static void main(String[] args) {
        int sum = 0;
        for (int i = 1; i <= 10; i++) {
            sum += i;
        }
        System.out.println(sum);
    }
}

```

Задача 1146 «Суммирование»: сумма целых чисел из входного потока.

```

package timus.task_1146;

import java.util.Scanner;

```

```

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        long sum = 0L;

        while (scanner.hasNextLong()) {
            sum += scanner.nextLong();
        }

        System.out.println(sum);
        scanner.close();
    }
}

```

4. Вывод

В ходе выполнения лабораторной работы изучены основы многопоточного программирования в Java. Реализованы примеры создания потоков, синхронизации с помощью ReentrantLock и Condition, а также параллельная обработка массивов с количеством потоков, равным числу ядер процессора. Программа успешно компилируется и выполняется. Дополнительно решены задачи 1082 и 1146 с сайта acm.timus.ru.

5. Результаты выполнения программы

(демонстрационный режим: сокращённые задержки для отчёта)

=== Лабораторная работа №6: Многопоточность ===

--- Примеры решения задач ---

Пример 1. Два потока выводят имя и время

t1 12:57:29

t2 12:57:29

t1 12:57:30

t2 12:57:30

t1 12:57:31

t2 12:57:31

t2 12:57:32

t1 12:57:32

t1 12:57:33

t2 12:57:33

t2 12:57:34

t1 12:57:34

t1 12:57:35

t2 12:57:35

t1 12:57:36

t2 12:57:36

t1 12:57:37

t2 12:57:37

t2 12:57:38

t1 12:57:38

Пример 2. Поток выводит числа от 1 до 10

1

2

3

4

5

6

7

8

9

10

Пример 3. Чётные и нечётные числа в двух потоках

Нечётное: 1

Чётное: 2

Нечётное: 3

Чётное: 4

Нечётное: 5

Чётное: 6

Нечётное: 7

Чётное: 8

Нечётное: 9

Чётное: 10

--- Задания для самостоятельной работы ---

Задание 1. Два потока выводят имя и время

t2 12:57:49

t1 12:57:49

t2 12:57:50

t1 12:57:50

t2 12:57:51

t1 12:57:51

t2 12:57:52

t1 12:57:52

t2 12:57:53

t1 12:57:53

t2 12:57:54

t1 12:57:54

t2 12:57:55

t1 12:57:55

t2 12:57:56

t1 12:57:56

t2 12:57:57

t1 12:57:57

t2 12:57:58

t1 12:57:58

Задание 2. Поток выводит числа от 1 до 10

1

2

3

4

5

6

7

8

9

10

Задание 3. Чётные и нечётные числа в двух потоках

Нечётное: 1

Чётное: 2

Нечётное: 3

Чётное: 4

Нечётное: 5

Чётное: 6

Нечётное: 7

Чётное: 8

Нечётное: 9

Чётное: 10

Задание 4. Десять потоков выводят свой номер

Поток №1

Поток №2

Поток №6

Поток №7

Поток №4

Поток №3

Поток №9

Поток №8

Поток №5

Поток №10

Задание 5. Максимальный элемент массива

Массив: [12, -4, 87, 5, 33, 87, 0, -19, 44, 7]

Количество потоков: 12

Максимум: 87

Задание 6. Сумма элементов массива

Массив: [12, -4, 87, 5, 33, 87, 0, -19, 44, 7]

Количество потоков: 12

Сумма: 252